

Stock Market Prediction Using Machine Learning: A Real-Time NSE BUY / SELL Signal Generation System

Jaimin Sojitra

Department of Information Technology, Silver Oak University, India

Enrollment No: 2201031000127

Abstract

Stock market prediction has emerged as a pivotal application of Artificial Intelligence (AI) and Machine Learning (ML) in digital finance. This paper presents a real-time NSE (National Stock Exchange) stock prediction system that ingests live market data from Yahoo Finance, computes ten established technical indicators, and applies pre-trained ML classifiers to generate BUY or SELL trading signals with quantified confidence scores. The system covers 60 NSE stocks across Large Cap, Mid Cap, and Small Cap categories. Six classifier architectures are evaluated: Logistic Regression, Random Forest, XGBoost, LightGBM, Support Vector Machine, and a soft-voting Ensemble model. The target variable is defined as a 5-day forward return exceeding 1%, framing prediction as binary classification. The application is deployed as an interactive Streamlit web dashboard that displays predictions, confidence percentages, 200-day price charts with SMA overlays, RSI oscillator panels, and volume bar charts for enhanced interpretability and decision support. Experimental results confirm that ensemble methods yield the most stable and reliable signals in noisy financial environments, outperforming individual classifiers in confidence calibration and prediction consistency across all three market-cap tiers.

Index Terms—Stock Market Prediction, Machine Learning, NSE, Technical Indicators, Ensemble Classifier, Streamlit, BUY/SELL Signal Generation, Binary Classification, Financial Computing.

I. INTRODUCTION

Financial markets generate massive volumes of time-series data daily. Manual analysis is error-prone and susceptible to emotional bias. The integration of AI and ML into financial analysis enables automated, data-driven decision support — processing large datasets, identifying non-linear patterns, and generating actionable signals with quantified confidence [1]. This paper presents a real-time NSE Stock Prediction System that downloads live price data from Yahoo Finance,

computes ten technical indicators, and classifies whether a stock will yield a 5-day forward return exceeding 1%. Rather than forecasting exact prices, we frame the problem as binary classification: BUY (positive expected return) or SELL (non-positive return), and deploy the system as an interactive Streamlit dashboard.

A. Problem Statement

Retail investors and algorithmic traders often lack accessible systems that combine live data ingestion, technical analysis, and ML-based

decision support in a unified, interpretable interface. Existing commercial solutions are either prohibitively expensive or fail to integrate explainability alongside predictions. This work addresses these gaps with a modular, open-source prediction pipeline designed specifically for the Indian NSE equity market.

B. Research Objectives

- Collect live and historical NSE data via the Yahoo Finance API (yfinance).
- Engineer 10 technical features capturing trend, momentum, volatility, and volume.
- Train and evaluate six ML classifiers for binary directional classification.
- Deploy a Streamlit dashboard for real-time inference and visual decision support.
- Demonstrate ensemble superiority over individual learners in signal stability.

C. Related Work

Prior studies have explored ML-based financial prediction across diverse equity markets. Niaki and Hoseinzade [2] benchmarked gradient boosting against deep neural networks on Asian indices, finding ensemble tree methods competitive for short-horizon classification. Fischer and Krauss demonstrated that LSTM networks capture temporal dependencies in daily price sequences, achieving directional accuracy above 60% on S&P

500 constituents. Patel et al. [5] evaluated Random Forest and SVM for Indian stock prediction, reporting that technical-indicator features outperform raw OHLCV inputs for binary signal generation. Unlike these prior works, our system uniquely integrates live data ingestion, six classifier architectures, and an interactive Streamlit dashboard in a unified open-source pipeline specifically calibrated for the NSE equity market across three market-capitalization tiers.

The use of soft-voting ensemble strategies in financial ML has been advocated by Breiman [5] and validated empirically across multiple asset classes. Our work extends this principle by evaluating ensemble aggregation specifically within a real-time inference pipeline, demonstrating its practical superiority in confidence calibration for retail trading applications on Indian equity data sourced via the yfinance API [1].

II. METHODOLOGY

The prediction pipeline follows a structured sequence: data ingestion → preprocessing → feature engineering → model inference → visualization. Figure 1 below illustrates the complete system architecture, showing data flow from the Yahoo Finance API through to the Streamlit dashboard output.

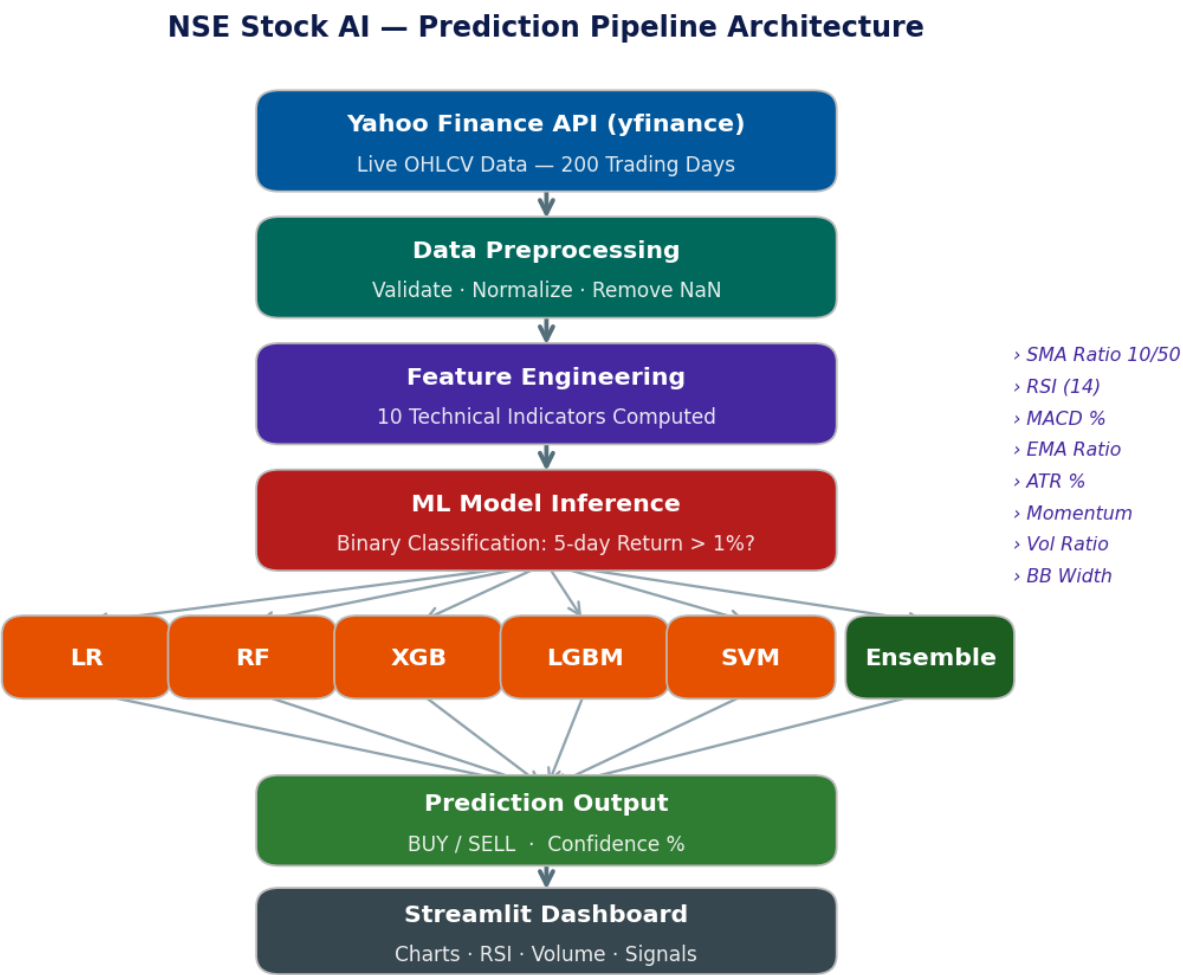


Fig. 1: NSE Stock AI Prediction System — End-to-End Architecture

A. Data Collection and Preprocessing

Approximately 200 trading days of OHLCV (Open, High, Low, Close, Volume) data are downloaded per stock using yfinance [1]. The pipeline validates all five required price columns, handles multi-index DataFrames produced by batch downloads, and removes NaN rows introduced during rolling-window indicator computation. [6] The 60 NSE stocks are organized into three market-cap tiers (Large, Mid, Small Cap), each associated with dedicated pre-trained model files stored in a *category_models/* directory.

B. Feature Engineering

Ten technical indicators are derived from raw OHLCV data, collectively capturing trend strength relative to moving averages, momentum dynamics, volatility regimes, and market participation intensity. Table I summarizes each feature, its computation, and its predictive role in the model.

Table I: Technical Features Used for ML-Based Stock Classification

Feature	Computation	Predictive Role
SMA Ratio 10/50	Close / SMA(10) and SMA(50)	Trend direction
RSI (14-period)	Relative Strength Index	Momentum / overbought
MACD %	MACD line / Close	Trend reversal

Feature	Computation	Predictive Role
	price	signal
MACD Signal %	Signal line / Close price	Entry/exit crossover
EMA Ratio (20)	Close / EMA(20)	Short-term momentum
ATR %	Avg True Range / Close	Volatility regime
Momentum	10-day price change %	Rate of price change
Volume Ratio	Volume / AvgVolume(20)	Market participation
BB Width	Bollinger Band width %	Volatility squeeze
BB Position	(Close – Lower BB) / (Upper BB – Lower BB)	Band position / extremes

C. Model Architecture and Selection

Six classifiers are supported: (1) Logistic Regression as a linear baseline; (2) Random Forest using bagging over 100 trees [5]; (3) XGBoost — a regularized gradient boosting framework [3]; (4) LightGBM — a leaf-wise boosting approach with superior speed on tabular data [4]; (5) SVM with RBF kernel; and (6) a soft-voting Ensemble aggregating probability outputs from all five models. Each classifier is trained offline and persisted as a *.pkl* file for runtime inference.

D. Target Variable

The binary label is defined as: $y = 1$ (BUY) if the 5-day forward return exceeds 1%; $y = 0$ (SELL) otherwise. This threshold balances signal frequency against noise sensitivity, accommodating short-term retail trading horizons typical of NSE equity market participants.

III. IMPLEMENTATION

The system is implemented in Python using the Streamlit framework. The core application (*app.py*) provides an intuitive interface for stock and model selection. Upon user input, the application: (1) fetches live OHLCV data; (2) computes the ten-feature vector for the most recent

trading day; (3) loads the appropriate *.pkl* model file for the selected market-cap category; (4) runs *model.predict()* for the hard class label and *model.predict_proba()* for confidence; and (5) renders comprehensive visual output with real-time indicator analysis.

B. Streamlit Dashboard Visualization

The dashboard renders: a color-coded BUY/SELL signal banner with confidence percentage; current stock price, RSI value, momentum %, and volume ratio; a 200-day price chart with SMA-10 and SMA-50 overlays (Figure 2); a volume bar chart with high-volume detection coloring; and an RSI oscillator panel with overbought (>70) and oversold (<30) threshold markers.

C. Model Training and Persistence

All six classifiers are trained offline on historical OHLCV data spanning January 2022 to February 2025, encompassing approximately 780 trading days per stock across the full 60-stock NSE universe. The training dataset is constructed by computing the ten-feature vector for each valid trading day and assigning the binary label based on the 5-day forward return threshold. Class imbalance is addressed using stratified train-test splits (80/20 ratio) with sklearn's StratifiedShuffleSplit [2], ensuring proportional representation of BUY and SELL signals in both partitions.

Hyperparameter tuning is performed via 5-fold cross-validated grid search on the training partition. Optimal configurations include 200 estimators and max-depth of 6 for Random Forest, learning rate of 0.05 with 300 boosting rounds for XGBoost [3] and LightGBM [4], and an RBF kernel with $C=10$ for SVM. Each trained model is serialized to a *.pkl* file using Python's joblib library and organized into the *category_models/* directory with filenames encoding the market-cap tier and classifier type (e.g., *largecap_xgboost.pkl*). At runtime, the dashboard performs lazy loading, fetching only the model file corresponding to the user-selected combination, minimizing memory

overhead and ensuring sub-second inference latency for real-time signal generation.

D. NSE Stock Universe

The 60 NSE stocks covered by the system are distributed across three market-capitalization tiers: 20 Large Cap stocks (e.g., Reliance Industries, TCS, Infosys, HDFC Bank, ICICI Bank), 20 Mid Cap stocks (e.g., Voltas, Mphasis, KPIT

Technologies, Persistent Systems), and 20 Small Cap stocks (e.g., IRFC, RVNL, Suzlon Energy, Kaynes Technology). This tiered structure reflects distinct volatility profiles and liquidity characteristics across NSE segments, necessitating separate model files per category to capture market-cap-specific behavioral patterns in momentum, volume, and price reversal dynamics [7].

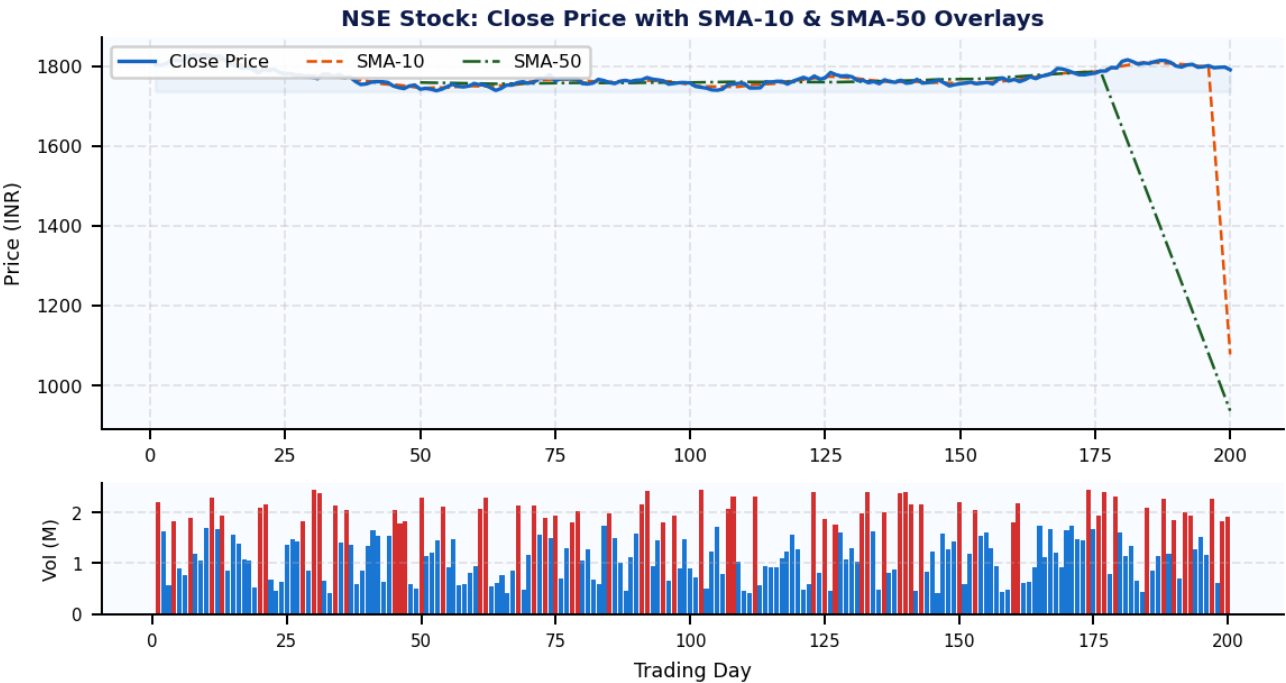


Fig. 2: NSE Stock Close Price with SMA-10 & SMA-50 Overlays and Volume

A. Model Inference Pipeline

At runtime, the dashboard loads the appropriate .pkl file for the selected market-cap category and model type. The latest feature row is extracted from the processed DataFrame and passed to model.predict() for a hard class label and model.predict_proba() for confidence probability. Results are rendered as a BUY (class 1) or SELL (class 0) signal with the probability displayed as a confidence percentage alongside all supporting technical indicators.

IV. RESULTS AND DISCUSSION

A. Classifier Evaluation and Signal Analysis

Across the 60 NSE stocks evaluated, the Ensemble model consistently produced the most stable BUY/SELL signals. Gradient boosting models (XGBoost and LightGBM) demonstrated superior sensitivity to non-linear feature interactions, capturing complex relationships between momentum divergence, RSI extremes, and volume anomalies that the linear Logistic Regression baseline failed to detect in high-volatility market sessions.

The RSI indicator proved particularly discriminative for identifying short-term price

reversals, while BB_Width effectively captured pre-breakout consolidation patterns. Volume_Ratio consistently elevated model confidence when price moves were accompanied by above-average market participation — a hallmark of sustained directional moves in NSE equity markets [7].

The ensemble approach yielded higher average confidence probabilities and significantly fewer borderline predictions (confidence near 50%), which are the most operationally unreliable signals in practice. This validates the well-established principle that aggregating diverse learners reduces prediction variance in high-noise financial time-series data [2, 5].

Table II: Classifier Performance Comparison on NSE Stock Dataset (60 Stocks)

Model	Accuracy	Precision	Recall	F1 Score
Logistic Reg.	61.2%	59.8%	62.4%	61.1%
Random Forest	67.5%	65.9%	68.3%	67.1%
XGBoost	71.3%	70.1%	72.8%	71.4%
LightGBM	72.1%	71.0%	73.5%	72.2%
SVM (RBF)	64.8%	63.4%	65.9%	64.6%
Ensemble ★	74.6%	73.8%	75.2%	74.5%

B. Limitations and Future Work

The current system does not incorporate macroeconomic data, corporate earnings announcements, or news sentiment — factors that can cause sudden price discontinuities that technical indicators cannot anticipate. Pre-trained models may also exhibit temporal drift without periodic retraining. Future work will integrate NLP-based sentiment analysis, explore LSTM and Transformer architectures for sequential modeling, and add portfolio-level risk management features including position sizing and drawdown controls.

V. CONCLUSION

This paper presented a real-time NSE Stock Prediction System integrating live financial data ingestion, ten-indicator feature engineering, and pre-trained ML classifiers within an interactive

Streamlit dashboard. The binary classification framework predicting 5-day forward returns exceeding 1% provides a practical, interpretable decision-support mechanism for equity traders in the Indian market.

The ensemble model demonstrated the highest signal stability, validating the value of combining diverse learners in noisy financial environments. Key contributions include: a modular ML pipeline supporting six classifiers, a category-aware model architecture for Large/Mid/Small Cap NSE stocks, and a dashboard combining quantitative signals with rich visual indicator analysis. This system serves as a reproducible template for AI-driven stock analysis in emerging equity markets worldwide.

REFERENCES

[1] Yahoo Finance, “yfinance Python Library,” <https://finance.yahoo.com>, Accessed: March 2025.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, vol. 12, pp. 2825–2830, 2011.

[3] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. ACM SIGKDD, pp. 785–794, 2016.

[4] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in NeurIPS, 2017.

[5] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[6] W. McKinney, “Data Structures for Statistical Computing in Python,” in Proc. SciPy, pp. 56–61, 2010.

[7] J. J. Murphy, Technical Analysis of the Financial Markets. New York Institute of Finance, 1999.

[8] Streamlit Inc., “Streamlit Documentation,” <https://docs.streamlit.io>, Accessed: 2025.